

Temă seminar - Algoritmi Aproximativi

• Knapsack

1. Fie S un set de numere naturale $\{s_1, s_2, \dots, s_n\}$ și K un număr natural, cu $K \geq s_i$ pentru orice i între 1 și n . Vrem să determinăm suma maximă, dar care să fie $\leq K$, ce poate fi formată din elementele din S (numerele pot fi luate cel mult o singură dată).

a) Scrieți un algoritm pseudo-polinomial care găsește suma optimă. Indicați complexitatea de timp/spațiu a algoritmului propus de voi și justificați de ce acesta este corect (de ce soluția găsită este optimă).

Rezolvare:

- Pași : • inițializare :

$$dp[0] = 1$$

$$dp[1] = dp[2] = \dots = dp[K] = 0$$

• Transiția :

Când procesăm un element s_i , orice sumă veche s pentru care $dp[s] = 1$ ne permite să obținem suma : $s + s_i$, doar dacă $s + s_i \leq K$

• Parcurgerea descrescătoare : Trebuie să parcurgem sumele de la K în jos pentru că fiecare element

poate fi folosit cel mult o singură dată.

- Pseudocod

SubsetSum($S[1..n]$, K):

$dp[1..K] \leftarrow 0$

$dp[0] \leftarrow 1$

pentru $i \leftarrow 1..n$:

 pentru $s \leftarrow K$ descrescator până la 0:

 dacă $dp[s] = 1$ și $s + S[i] \leq K$ atunci:

$dp[s + S[i]] \leftarrow 1$

 pentru $s \leftarrow K$ descrescator până la 0:

 dacă $dp[s] = 1$ atunci:

 return s

- Corectitudinea

• invariant

După ce am procesat primele i elemente, avem:

$dp[s] = 1 \Leftrightarrow$ există o submulțime din $\{s_1, \dots, s_i\}$
cu sumă exact s

• Baza inducției

Pentru $i = 0$:

- putem obține doar suma 0
- nu putem obține nimic altceva

$\Rightarrow$ deci invariantul este adărat

• Pas inductiv

Presupunem că invariantul este adevărat pentru primele $i-1$ elemente.

Când procesăm S_i , pentru fiecare sumă s deja realizabilă:

- fie nu folosim S_i și atunci s rămâne realizabilă
- fie folosim S_i și atunci obținem suma $s + S_i$.

Deci exact toate sumele realizabile cu primele i elemente sunt marcate.

Prin urmare, invariantul rămâne adevărat.

• Final

După procesarea tuturor elementelor, $dp[s] = 1$ exact pentru sumele realizabile din întregul set.

Dacă alegem cel mai mare $s \leq K$ cu $dp[s] = 1$, acesta este exact suma optimă cerută.

- Complexitate

$$\begin{cases} \text{timp} = O(n \cdot K) \\ \text{spațiu} = O(K) \end{cases}$$

Aveam :

- n elemente

- pentru fiecare parcurgem cel mult K posibilități

h) Scrieți un algoritm aproximativ care calculează o sumă cel puțin pe jumătate de mare ca cea optimă, dar rulează în timp $O(n)$ și complexitate spațiu $O(1)$.

Rezoluare: - Pași:

1. Căutăm elementul maxim din S .
2. Dacă acel element $\geq K/2$, îl întoarcem.
3. Altfel, toate elementele $\in K/2$.

Atunci adunăm elementele pe rând până când:

- fie depășim $K/2$, caz în care acea sumă este răspunsul
- fie terminăm toate elementele, caz în care suma totală este răspunsul

- Pseudocod

HalfApprox ($S[1..n], K$):

max ← 0

pentru $i \in 1..n$:

dacă $S[i] > \text{max}$ atunci

max ← $S[i]$

dacă max $\geq K/2$ atunci

return max

sum ← 0

pentru $i \in 1..n$:

sum ← sum + $S[i]$

dacă sum $\geq K/2$ atunci

return sum

- Corectitudinea : suma returnata de algoritmul nu depăsește micsoalata K .

• Caz 1: algoritmul întoarce max

Avem : $max \leq K$ deoarece în enunț se precizează că $K \geq S_i$ pentru orice i .

Deci max este soluție fezabilă.

• Caz 2: algoritmul întoarce o suma sum

$max < \frac{K}{2} \Rightarrow$ fiecare element din sir $< \frac{K}{2}$

Algoritmul se oprește la primul moment când : $sum \geq \frac{K}{2}$

Notăm cu sum_{old} suma de dinaintea de adăugarea ultimului element : $sum_{old} < \frac{K}{2}$

$$sum = sum_{old} + S_i < \frac{K}{2} + \frac{K}{2} = K$$

$\Rightarrow$ Suma este fezabilă

Dacă algoritmul nu ajunge micsoalata la $K/2$, atunci întoarce suma tuturor elementelor, care este de asemenea fezabilă, pentru că dacă ar fi fost $> K$, atunci pe parcurs am fi trecut de $K/2$.

- Demonstratie factor de aproximare

OPT = valoare soluție optime

Orice soluție fezabilă are suma cel mult K :

$$OPT \leq K$$

Trbuie să arătăm că algoritmul întoarce o valoare cel puțin egală cu: $\frac{OPT}{2}$

• Cazul 1: există un element $S_i \geq \frac{K}{2}$

$$ALG = \max \geq \frac{K}{2}$$

$$\text{Dar cum } OPT \leq K : \frac{K}{2} \geq \frac{OPT}{2}$$

$$\Rightarrow ALG \geq \frac{OPT}{2}$$

• Cazul 2 : toate elementele sunt $< \frac{K}{2}$

2.1 : pe parcurs se dă o sumă $\text{sum} \geq \frac{K}{2}$

$$\Rightarrow ALG = \text{sum} \geq \frac{K}{2}$$

$$\text{Cum } OPT \leq K : ALG \geq \frac{OPT}{2}$$

2.2 : nu ajunge niciodată la $K/2$

$$ALG = S_1 + S_2 + \dots + S_n$$

Acesta este chiar suma maximă a poate folosite
din elementele date: $ALG = OPT$

$$\Rightarrow ALG \geq \frac{OPT}{2}$$

$\Rightarrow$ Algoritmul este $\frac{1}{2}$ - aproximativ

- Complexitate

$$\text{temp} = O(n)$$

$$\text{spațiu} = O(1) \quad (\text{max și sum})$$

• Load Balance

1. Este o iterație a problemei Load Balancing pentru 2 mașini.
La seminarul de algoritmi aproximativi unul dintre studenți propune un algoritm de rezoluare și susține că acesta este 1.1 aproximativ.
El rulează algoritmul pe un set de n activități și obține o încărcătură de 80 pe una dintre mașini, respectiv 120 pe cealaltă. Este posibil ca factorul lui de aproximare să fie corect...

Rezoluare Avem: - un set de n activități
- fiecare activitate are un timp de execuție t_i

Algoritmul: - este rulat o singură dată
- parcurge activitățile
- le distribuie pe cele 2 mașini

$$ALG = 120$$

$$1.1 - \text{aproximativ} \Rightarrow ALG \leq 1.1 \cdot OPT \Rightarrow OPT \geq \frac{ALG}{1.1} = \frac{120}{1.1} \approx 109.09$$

Apesar, dacă afirmația lui este corectă, atunci pentru această instanță optimul trebuie să fie cel puțin 110.

Din curs, optim lower bound-ul general:

$$OPT \geq \max \left(\frac{1}{m} \sum_{1 \leq j \leq n} t_j, \max \{ t_j \mid 1 \leq j \leq n \} \right)$$

$m = \text{nr. mașini}$
primul termen = timpul mediu de lucru al tuturor mașinilor pentru a efectua toate taskurile
al doilea termen = timpul de lucru al celei mai costisitoare activități

$$m = 2 \Rightarrow \text{media} = \frac{80 + 120}{2} = 100$$

$$\Rightarrow OPT \geq 100$$

dar pentru 1.1 ar trebui chiar: $OPT \geq 103.09$

a) .. ținând cont că rezultatul obținut anterior a fost făcut pe un set de activități, fiecare cu timpul de lucru cel mult 100?

Rezoluare: Trebuie să verificăm dacă există o constantă:

- cu toate joburile ≤ 100
- pentru care algoritmul obține încărcăturile 120×80
- cu optimul suficient de mare încât factorul 1.1 să nu fie contrazis

Ex: $(80, 40, 40, 40)$

$\hookrightarrow$ toate joburile ≤ 100

Dacă aplicăm algoritmul greedy clasic de list scheduling pe 2 mașini, care pune fiecare job pe mașina momentan cea mai puțin încărcată obținem:

- $j_1 = 80$: $(80, 0)$
- $j_2 = 40$: $(80, 40)$
- $j_3 = 40$: $(80, 80)$
- $j_4 = 40$: $(120, 80)$

$$\text{Suma totală} = 80 + 3 \times 40 = 200$$

Pentru 2 mașini o încărcătură ideală ar fi în jur de 100

pe 100.

Dar cu aceste joburi, sumele posibile ale submultimilor sunt: 0, 40, 80, 120, 160, 200.

Am putea detine o suma 100 și nu 110. Astfel, nu există o împărțire mai mică decât 120.

$$\rightarrow OPT = 120$$

$$\frac{ALG}{OPT} = \frac{120}{120} = 1 \leq 1.1 \Rightarrow DA$$

b) ... ținând cont că rezultatul obținut anterior a fost făcut pe un set de activități, fiecare cu timpul de lucru cel mult 10?

Rezolvare: Notăm:

- h = mașina care ajunge la load maxim final
- g = ultimul job atribuit mașinii h
- $load'(h)$ = încărcarea mașinii înainte de adăugarea lui g
- L = încărcarea celeilalte mașini

$$load'(h) \leq L$$

$\Rightarrow$ încărcarea finală a mașinii h : $load'(h) + t_g$

$$load'(h) \leq L \Rightarrow load'(h) - L \leq 0 \quad | + t_g \Rightarrow$$

$$\Rightarrow (load'(h) - L) + t_g \leq t_g \Rightarrow \underbrace{(load'(h) + t_g)}_{\text{încărcare finală } m_1} - L \leq t_g \quad \underbrace{- L}_{m_2} \leq t_g \quad g$$

Prin urmare, în cazul a 2 mașini:

diferența finală $\leq$ durata ultimului job

Dacă fiecare activitate are timp cel mult 10, rezultă:

$$|L_1 - L_2| \leq 10$$

Dar în enunț, diferența este: $120 - 80 = 40$ } contradicție

$\Rightarrow$ NU

3. Fie algoritmul Ordered-Scheduling Algorithm, care implică algoritmul descris anterior la care adăugăm o preprocesare cu care sortăm descrescător activitățile după timpul de desfășurare. Th. 2 afirmă că acest algoritmul $\frac{3}{2}$ -aproximativ. Arătați că acest factor de aproximare poate fi îmbunătățit la $\frac{3}{2} - \frac{1}{2^m}$ (unde m este numărul de calculatoare pe care se pot executa activitățile).

Rezolvare - Din curs avem:

Teorema 1 : $ALG \leq (2 - \frac{1}{2^m}) OPT$

Lema 3 : $OPT \geq t_m + t_{m+1}$

unde $t_1 \geq t_2 \geq \dots \geq t_n$

(timpuri sortate descrescător)

- Notatii :
- h = mașina cu încărcarea finală maximă
 - j = ultimul job atribuit mașinii h

• $\text{load}'(h)$ = încărcarea mașinii h înainte de a primi jobul J

$$\Rightarrow ALG = \text{load}'(h) + t_J = \text{load}(h)$$

$$t_1 \geq t_2 \geq \dots \geq t_n$$

- Pași:

• Pasul 1 - estimarea lui $\text{load}'(h)$

Algoritmul atribuie fiecare job mașinii momentan cel mai puțin încărcată, mașina h era cel mai puțin încărcată înainte de a primi J :

$\rightarrow$ suma tuturor joburilor puse înainte de J

$$\text{load}'(h) \leq \underbrace{\frac{1}{m} \sum_{j \in J} t_j}_{\text{media}} \quad (\Rightarrow)$$

$$\Leftrightarrow \text{load}'(h) \leq \frac{1}{m} \left(\sum_{j=1}^n t_j - t_J \right) + t_J \quad (\Rightarrow)$$

$$\Leftrightarrow \underbrace{\text{load}'(h) + t_J}_{ALG} \leq \frac{1}{m} \left(\sum_{j=1}^n t_j - t_J \right) + t_J \quad (\Rightarrow)$$

$$\Leftrightarrow ALG \leq \frac{1}{m} \sum_{j=1}^n t_j + t_J \left(1 - \frac{1}{m} \right) \quad \Rightarrow$$

Lemma 1: $OPT \geq \frac{1}{m} \sum_{j=1}^n t_j$

$$\Rightarrow ALG \leq OPT + t_J \left(1 - \frac{1}{m} \right)$$

- Pasul 2 - joburile sunt sortate

În Order Scheduling, activitățile sunt ordonate descrescator.

Dacă $p \leq m$, atunci jobul p este unul dintre primele m joburi și va fi pus pe o mașină goală. În acest caz:

$$ALG = t_p \leq t_1 \leq OPT$$

Dacă $p > m$, atunci: $t_p \leq t_{m+1}$ (fiindcă șirul este sortat descrescator)
 $\hookrightarrow t_1 \geq t_2 \geq \dots \geq t_m \geq t_{m+1} \geq \dots$

- Pasul 3 - folosim Lemma 3

$$\left. \begin{array}{l} \text{Din curs avem: } OPT \geq t_m + t_{m+1} \\ t_m \geq t_{m+1} \end{array} \right\} \Rightarrow OPT \geq 2 t_{m+1}$$

$$\Rightarrow t_{m+1} \leq \frac{OPT}{2}$$

$$\text{Cum } t_p \leq t_{m+1} \Rightarrow t_p \leq \frac{OPT}{2}$$

- Pasul 4 - înlocuim în formula generală

$$\left. \begin{array}{l} ALG \leq OPT + t_p \left(1 - \frac{1}{m}\right) \\ t_p \leq \frac{OPT}{2} \end{array} \right\} \Rightarrow ALG \leq OPT + \left(1 - \frac{1}{m}\right) \frac{OPT}{2} \Rightarrow$$

$$\Rightarrow ALG \leq OPT + \frac{OPT}{2} - \frac{OPT}{2m} \Rightarrow ALG \leq \left(\frac{3}{2} - \frac{1}{2m}\right) OPT$$

$\Rightarrow$ Algoritmul este $(\frac{3}{2} - \frac{1}{2m})$ - aproximativ

• Travelling Salesman Problem

1. Considerăm o variantă a TSP, în care toate muchiile din graf au ponderea 1 sau 2.

a) Arătați că problema rămâne NP-hard pentru aceste instanțe.

Rezolvare: Fie $G=(V, E)$ un graf simplu neponderat, cu $|V|=n$.

Construim un graf complet ponderat $G'=(V, E')$ pe aceeași mulțime de vârfuri, astfel:

$$w(x, y) = \begin{cases} 1, & \text{dacă } (x, y) \in E \\ 2, & \text{dacă } (x, y) \notin E \end{cases}$$

Această construcție se face în timp polinomial.

• Cazul 1: G are ciclu Hamiltonian

Dacă G are ciclu Hamiltonian, atunci acel ciclu folosește doar muchii din E , deci în G' toate muchiile lui au cost 1.

Prin urmare, există un tur TSP în G' de cost: n

Deci: $OPT = n$.

• Cazul 2: G nu are ciclu

Dacă G nu are ciclu Hamiltonian, atunci orice ciclu Hamiltonian din G' trebuie să folosească cel puțin o muchie

care nu era în E , deci de cost 2.

Restul muchiilor au cost cel puțin 1, deci orice tur are cost cel puțin:

$$(n-1) \cdot 1 + 2 = n+1$$

$$\text{Deci: } OPT \geq n+1$$

$\Rightarrow$ Prin urmare, dacă am avea un algoritm polinomial care rezolvă exact această variantă de TSP, am putea decide în timp polinomial dacă G are sau nu ciclu Hamiltonian:

- calculăm optimele în G'
- dacă rezultatul este n , atunci G este Hamiltonian, altfel, nu este

Dar Hamilton Cycle Problem este NP-completă, deci problema este NP-hard.

a) Arătați că aceste ponderi satisfac cu continuare inegalitatea triunghiulară ($w(u, v) \leq w(u, w) + w(w, v)$)

Rezolvare: Cum fiecare muchie are cost 1 sau 2, orice muchie are cost cel mult 2. Pe de altă parte, suma costurilor a două muchii este cel puțin $1+1=2$. Rezultă că pentru orice 3 vrfuri u, v, w :

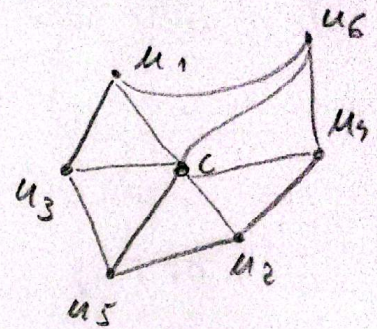
$$w(u, v) \leq 2 \leq w(u, w) + w(w, v)$$

$\Rightarrow$ deci respectă inegalitatea triunghiulară.

c) Algoritmul descris în curs este un algoritm 2-aproximativ pentru forma generală a TSP (pentru instanțele care respectă inegalitatea triunghiulară). Verificați dacă în această variantă a problemei, algoritmul din curs este chiar $\frac{5}{2}$ -aproximativ.

Rezolvare: Algoritmul din curs pentru TSP este cel descris pe:

1. calcularea unui MST
2. alegerea unei rădăcini
3. parcurgerea DFS a arborelui
4. închiderea ciclului



(toate muchiile de cost 1)

Contraexemplu:

Considerăm un graf complet ponderat pe mulțimea de vârfuri $V = \{c, u_1, u_2, u_3, u_4, u_5, u_6\}$ și definim ponderile astfel:

- muchiile (c, u_i) , $i = \overline{1, 6}$
- $(u_1, u_3), (u_3, u_5), (u_5, u_2), (u_2, u_4), (u_4, u_6), (u_6, u_1)$

sau cost 1

- restul au cost 2

1. instanța respectă inegalitatea triunghiulară

$$\Rightarrow w(x, y) \leq 2 \leq w(x, z) + w(z, y)$$

Prin urmare, instanța este metrică

2. Există ciclu Hamiltonian de cost 7

Considerăm ciclul: $\{c, u_1, u_3, u_5, u_2, u_4, u_6, c\}$ } $\Rightarrow$

Toate muchiile au cost 1.

$$\Rightarrow \text{cost ciclul} = 1 + 1 + 1 + 1 + 1 + 1 + 1 = 7 \Rightarrow OPT \leq 7 \quad (1)$$

Pe de altă parte, orice ciclu Hamiltonian pe 7 vârfuri are exact 7 muchii, iar fiecare muchie are cost cel puțin 1. $\Rightarrow OPT \geq 7 \quad (2)$

$$\stackrel{(1), (2)}{\implies} OPT = 7$$

3. Algoritm

Aligem ca MST steaua cu centru în c :

$$T = \{(c, u_1), (c, u_2), (c, u_3), (c, u_4), (c, u_5), (c, u_6)\}$$

$$\Rightarrow MST = 6 \quad (\text{toate muchiile au cost 1})$$

Rădăcina arborelui este c și considerăm parcurgerea DFS:

$$c, u_1, u_2, u_3, u_4, u_5, u_6$$

Atunci ciclul este:

$$\Gamma = (c, u_1, u_2, u_3, u_4, u_5, u_6, c)$$

4. Costul soluției

$$w(c, u_1) = w(u_6, c) = 1$$

$$w(u_1, u_2) = w(u_2, u_3) = w(u_3, u_4) = w(u_4, u_5) = w(u_5, u_6) = 2$$

$$\Rightarrow ALG = 12$$

$$\frac{ALG}{OPT} = \frac{12}{7} \approx 1.714 > \frac{3}{2} \Rightarrow ALG > \frac{3}{2} OPT \Rightarrow \text{Algoritmul nu este } \frac{3}{2} \text{-aproximativ.}$$

• Vertex Cover

Fie $X = \{x_1, x_2, \dots, x_n\}$ o multime de variabile booleane.

Numim formula booleană (peste multimea X) în CNF o expresie de forma $C_1 \vee C_2 \vee \dots \vee C_m$ unde fiecare predicat (clausă) C_i este o disjuncție a unui număr de variabile. Evident că orice astfel de expresie va fi evaluată ca true dacă toate elementele din X iau valoarea true. Ne interesează în schimb să aflăm numărul min de elemente din X care trebuie să aibă valoarea true a.ș. toată expresia să fie true.

Algoritm (3CNF) $\rightarrow$ Greedy - 3CNF:

1. Fie $C = \{C_1, \dots, C_m\}$ multimea de predicate, $X = \{x_1, \dots, x_n\}$ multimea de variabile.

2. Cat timp $C \neq \emptyset$ execută:

(a) Alegeți aleator $C_j \in C$

(b) Fie x_i una dintre variabilele din C_j .

(c) $x_i \leftarrow \text{true}$

(d) Eliminați din C toate predicatele care îl conțin pe x_i .

3. Soluția constă din variabilele pe care le-am setat ca true pe parcursul execuției programului.

a) Este algoritmul descris mai sus un algoritm aproximativ? În cazul afirmativ, determinați factorul de aproximare. Alfel, justificați de ce nu este aproximativ.

Rezolvare :

Algoritmul este:

1. alegeți o clausă C_j

2. alegeți singura variabilă x_i din ea

3. $x_i \leftarrow \text{true}$

4. eliminați toate clauzele care îl conțin pe x_i

$\Rightarrow$ Algoritmul dat nu este p -aproximativ pentru nicio constantă p . Poate furniza un răspuns arbitrar de slab.

Demonstratie:

Considerăm formula:

$$(x_1 \vee y_1 \vee z) \wedge (x_2 \vee y_2 \vee z) \wedge \dots \wedge (x_h \vee y_h \vee z)$$

adică avem h clauze și fiecare conține variabila z .

Soluția optimă: Dacă alegem $z = \text{true}$. $\Rightarrow$ toate clauzele sunt adevărate.

$$\Rightarrow \text{OPT} = 1$$

Ce poate face algoritmul:

La fiecare pas, algoritmul poate alege din clauza curentă cu variabila z , o una dintre celelalte, de exemplu x_i .

Astfel, poate alege succesiv: $x_1, x_2, \dots, x_h$.

Fiecare alegere elimină doar clauza respectivă $\Rightarrow$

$$\Rightarrow \text{ALG} = h$$

$$\frac{\text{ALG}}{\text{OPT}} = \frac{h}{1} = h$$

Cum h poate fi oricât de mare, raportul nu este mărginit superior de nicio constantă.

$\Rightarrow$ Algoritmul nu este aproximativ.

b) Modificati algoritmul de mai sus a.o. astfel sa fie 3-aproximativ pentru problema initiala (si justificati de ce se obtine acest factor in urma modificarilor).

Rezolvare

- Approx-3CHF

1. $C = \{C_1, \dots, C_m\}$ multimea clauselor

2. $S = \emptyset$

3. Cat timp $C \neq \emptyset$:

• alegem o clausa $C_j = (x_a \vee x_b \vee x_c)$

• adaugam toate cele 3 variabile ca solutie:

$$S \leftarrow S \cup \{x_a, x_b, x_c\}$$

• eliminam din C toate clausule care contin cel putin una dintre variabilele x_a, x_b, x_c

4. La final, setam true toate variabilele din S

- Fecalibilitate: la fiecare pas, daca eliminam o clausa, asta inseamna ca ea contine cel putin o variabila din S care va fi setata ca true. $\Rightarrow$ la final, multimea clauselor devine vidă.

- Factor de aproximare

$C^* =$ multimea clauselor selectate de algoritmul

Este TO solutie fecalibila, adica o multime de variabile setate ca true.

Fiecare clausă din C^* trebuie satisfăcută, deci pentru fiecare clausă din C^* , soluția T trebuie să conțină cel puțin o variabilă din acea clausă.

Dar clauzele din C^* nu au variabile comune. Astfel, o singură variabilă aleasă din T poate „satisfice” cel mult o clausă din C^* .

$$\text{Deci: } |T| \geq |C^*| \Rightarrow \text{OPT} \geq |C^*|$$

La fiecare pas, pentru fiecare clausă selectată, algoritmul adaugă exact 3 variabile în S .

$$\left. \begin{array}{l} \Rightarrow \text{ALG} = |S| = 3|C^*| \\ \text{OPT} \geq |C^*| \end{array} \right\} \Rightarrow 3\text{OPT} \geq 3|C^*| = \text{ALG}$$

$\Rightarrow \text{ALG} \leq 3\text{OPT} \Rightarrow$ algoritmul este 3-aproximativ

c) Reformulați problema de mai sus sub formă unei probleme de programare liniară.

Rezolvare:

Folosim Weighted Vertex Cover:

- variabilele $x_i \in [0, 1]$
- funcție obiectiv de minimizare
- pentru fiecare obiect punem o constrângere liniară

Pentru fiecare variabilă booleană x_i , introducem o variabilă LP tot notată x_i , a.ș.

$$x_i = \begin{cases} 1, & \text{dacă } x_i \text{ este setată ca true} \\ 0, & \text{dacă } x_i \text{ este setată ca false} \end{cases}$$

Formulara LP : $\min \sum_{i=1}^n x_i$

sub constrangerile:

- pentru fiecare clausa : $C_j = (x_a \vee x_b \vee x_c)$

$$x_a + x_b + x_c \geq 1$$

- pentru toate variabilele:

$$0 \leq x_i \leq 1, \quad i = \overline{1, n}$$

— Explicatie:

Constrangerea $x_a + x_b + x_c \geq 1$ spune ca intr-o clausa trebuie macar una dintre cele 3 variabile sa fie true.

Daca am mai impune si $x_i \in \{0, 1\} \rightarrow$ problema discretă originală.

d) Dati o solutie s- aproximativă care să rezolve problema de programare liniară de la c).

Rezolvare: Algoritmul de rounding

Fie $(x_1^*, x_2^*, \dots, x_n^*)$ solutie optimă a relaxării LP.

Construim solutie discretă : $\hat{x}_i = \begin{cases} 1, & \text{dacă } x_i^* \geq \frac{1}{3} \\ 0, & \text{dacă } x_i^* < \frac{1}{3} \end{cases}$

Apoi setăm true toate variabilele pentru care $\hat{x}_i = 1$.

Factorul de aproximare

Costul soluției rotunjite este: $ALG = \sum_{i=1}^n \hat{x}_i$

Pentru orice i avem: $\hat{x}_i \leq 3x_i^*$

• dacă $x_i^* < 1/3 \Rightarrow \hat{x}_i = 0 \leq 3x_i^*$

• dacă $x_i^* \geq 1/3 \Rightarrow \hat{x}_i = 1 \leq 3x_i^*$

$$ALG = \sum_{i=1}^n \hat{x}_i \leq \sum_{i=1}^n 3x_i^* = 3 \sum_{i=1}^n x_i^*$$

Dar $\sum x_i^*$ este optimul relaxării LP, iar optimul LP este cel mult optimul problemei întregi, deoarece relaxarea are mai multe soluții admise.

$$\sum_{i=1}^n x_i^* \leq OPT \Rightarrow ALG \leq 3OPT$$